RK4 Attitude Propagator

Documentation for `src/RK4_Propogator.py`

Vivek Swamy

June 26, 2026

1 Overview

`RK4_Propogator.py` defines the **AttitudeSimulator** class, which is the top-level integrator that advances a rigid satellite's rotational state forward in time. It ties together two lower-level modules:

- `QuaternionKinematics` – computes $\dot{q}$ from the current attitude quaternion and angular velocity.
- `RotationalDynamics` – computes $\dot{\omega}$ from Euler's rigid body equation, given the inertia tensor and applied torque.

The state vector being propagated is

$$x = \begin{bmatrix} q \\ \omega \end{bmatrix}, \qquad q = \begin{bmatrix} q_0 & q_1 & q_2 & q_3 \end{bmatrix}^T \in \mathbb{R}^4, \quad \omega = \begin{bmatrix} \omega_x & \omega_y & \omega_z \end{bmatrix}^T \in \mathbb{R}^3.$$

The class integrates this coupled, nonlinear system using a classical **fourth-order Runge–Kutta (RK4)** scheme with a fixed timestep, and re-normalizes the quaternion after every step to suppress numerical drift away from the unit quaternion manifold.

2 Mathematical Background

2.1 Quaternion Kinematics

The attitude quaternion evolves according to

$$\dot{q} = \frac{1}{2}\,\Omega(\omega)\,q,$$

where $\Omega(\omega)$ is the skew-symmetric matrix

$$\Omega(\omega) = \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix}.$$

This is implemented in `QuaternionKinematics.quaternion_dot(q, omega)`.

2.2 Rotational Dynamics (Euler's Equation)

The angular velocity evolves according to Euler's rigid-body equation,

$$I\,\dot{\omega} = \tau_{\text{ext}} - \omega \times (I\,\omega),$$

so that

$$\dot{\omega} = I^{-1}\Big(\tau_{\text{ext}} - \omega \times (I\,\omega)\Big),$$

where I is the (constant, body-fixed) inertia tensor. This is implemented in `RotationalDynamics.omega_dot()`.

2.3 Coupled State Derivative

For the RK4 scheme, the combined derivative of the state is computed in one call:

$$f(q, \omega, \tau) = \begin{bmatrix} \dot{q} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} \frac{1}{2}\Omega(\omega)\,q \\ I^{-1}\big(\tau - \omega \times I\omega\big) \end{bmatrix}.$$

This corresponds to the private method `_state_derivative` in the code.

2.4 Why RK4 Instead of Euler Integration

The simplest possible integrator for $\dot{x} = f(x,t)$ is **explicit (forward) Euler**:

$$x_{n+1} = x_n + h\, f(x_n, t_n).$$

It was deliberately *not* used here, for the following reasons:

1. **Quaternion norm drift.** Euler only uses the derivative at the *start* of the step, so it consistently under- or over-shoots the true curved trajectory of q on the unit-quaternion manifold. In practice this means $\|q\|$ drifts away from 1 much faster under Euler than under RK4, requiring more frequent (and larger) renormalization corrections to mask an integration error that is fundamentally larger to begin with. RK4 samples the derivative at the start, twice at the midpoint, and at the end of the interval, so it tracks the curvature of $\dot{q} = \frac{1}{2}\Omega(\omega)q$ far more faithfully over each step.

2. **Coupled rotational dynamics.** $\dot{\omega}$ depends on the gyroscopic cross-coupling term $\omega \times I\omega$, which is nonlinear in ω. For satellites with even modest spin rates or inertia anisotropy, Euler integration of this term is prone to energy and angular-momentum drift and can become visibly unstable (oscillations growing in amplitude) at timesteps that RK4 integrates cleanly.

3. **Numerical stability margin.** Euler's stability region is small, so an unstable simulation is often the first symptom of a timestep that is simply too large, rather than a sign of a physical instability in the underlying system. RK4's larger stability region means `dt` can be chosen based on accuracy requirements rather than being forced down purely to keep Euler from diverging.

4. **Cost vs. accuracy trade-off is favorable.** RK4 costs 4× the derivative evaluations of Euler per step, but the derivative evaluations here (a 4×4 matrix-vector product and a 3×3 inertia solve) are cheap. The accuracy gained is large enough that, for a fixed error tolerance, RK4 with a larger `dt` is both more accurate *and* cheaper overall than Euler with the much smaller `dt` it would need to match that accuracy.

In short: Euler is cheap per step but accumulates error quickly and can destabilize the attitude/angular-velocity coupling; RK4 costs more per step but its much higher order of accuracy lets it use a larger, more practical `dt` while keeping the quaternion norm and angular momentum well-behaved over long simulation runs, which matters for a simulator meant to support controller tuning over tens of seconds to minutes of simulated time.

2.5 Fourth-Order Runge–Kutta Integration

Given state (q, ω) at time t, fixed timestep $h = \Delta t$, and (assumed constant over the step) torque τ, RK4 evaluates the derivative four times:

$$k_1 = f(q,\ \omega,\ \tau) \tag{1}$$

$$k_2 = f\Big(q + \tfrac{h}{2}k_1^{(q)},\ \ \omega + \tfrac{h}{2}k_1^{(\omega)},\ \tau\Big) \tag{2}$$

$$k_3 = f\Big(q + \tfrac{h}{2}k_2^{(q)},\ \ \omega + \tfrac{h}{2}k_2^{(\omega)},\ \tau\Big) \tag{3}$$

$$k_4 = f\Big(q + h\,k_3^{(q)},\ \ \omega + h\,k_3^{(\omega)},\ \tau\Big) \tag{4}$$

and combines them into the updated state:

$$q_{n+1} = q_n + \frac{h}{6}\Big(k_1^{(q)} + 2k_2^{(q)} + 2k_3^{(q)} + k_4^{(q)}\Big),$$

$$\omega_{n+1} = \omega_n + \frac{h}{6}\Big(k_1^{(\omega)} + 2k_2^{(\omega)} + 2k_3^{(\omega)} + k_4^{(\omega)}\Big).$$

Because RK4's intermediate stages perturb q away from the unit quaternion manifold , the propagator **renormalizes** q_{n+1} immediately after combining the stages:

$$q_{n+1} \leftarrow \frac{q_{n+1}}{\|q_{n+1}\|}.$$

This keeps $\|q\| = 1$ to machine precision at every logged step, preventing the orientation representation from drifting into a non-unit quaternion over long simulations.

3 Class Reference: `AttitudeSimulator`

3.1 Constructor

```
def __init__(self, inertia_tensor, q0, omega0, dt=0.01):
```

- **inertia_tensor** – 3×3 body-fixed inertia matrix, passed straight into `RotationalDynamics`, which pre-computes and caches its inverse.
- **q0** – initial attitude quaternion $[q_0, q_1, q_2, q_3]$.
- **omega0** – initial angular velocity $[\omega_x, \omega_y, \omega_z]$ (rad/s).
- **dt** – fixed integration timestep (s), default 0.01.

It initializes simulation time $t = 0$, creates a `history` dictionary with keys `"t"`, `"q"`, `"omega"`, and immediately logs the initial state via `_log()`.

3.2 `_state_derivative(q, omega, torque)`

Thin wrapper that calls into the kinematics and dynamics modules and returns the pair $(\dot{q}, \dot{\omega})$ used as a single RK4 stage evaluation.

3.3 `step(torque=np.zeros(3))`

Advances the simulation by exactly one timestep `dt`:

1. Computes the four RK4 stage pairs (k_1, k_2, k_3, k_4) for both q and ω, as derived in Section 3.4.
2. Combines them into `q_new` and `omega_new`.
3. Renormalizes `q_new` by its norm.
4. Updates `self.q`, `self.omega`, advances `self.t` by `dt`, and logs the new state.

Note that the same `torque` value is used across all four RK4 stages within a single call to `step` – i.e. torque is treated as piecewise-constant over each timestep, not re-evaluated at the intermediate stage times.

3.4 `_log()`

Appends the current `t`, a *copy* of `q`, and a *copy* of `omega` to the `history` dictionary. Copies are used (`.copy()`) so that later in-place updates to `self.q` / `self.omega` do not retroactively corrupt previously logged history entries.

3.5 `run(duration, torque_func=None)`

Runs the simulation forward for a given `duration` (s):

- Computes `steps = int(duration / self.dt)`.
- On each iteration, if a `torque_func(t, q, omega)` callback is supplied, it is evaluated at the *start* of the step to produce the torque applied during that step; otherwise zero torque is used.
- Calls `self.step(torque)` for each of the `steps` iterations.

This is the entry point intended for scripting – e.g., driving the satellite with an open-loop disturbance torque, or (once available) a closed-loop PID/reaction-wheel controller that computes `torque_func` from the current attitude error.

4 Full Source Listing

```
import numpy as np
from QuaternionKinematics import QuaternionKinematics
from RotationalDynamics import RotationalDynamics


class AttitudeSimulator:
    def __init__(self, inertia_tensor, q0, omega0, dt=0.01):
        self.kinematics = QuaternionKinematics()
        self.dynamics = RotationalDynamics(inertia_tensor)
        self.dt = dt

        self.q = np.array(q0, dtype=np.float64)
        self.omega = np.array(omega0, dtype=np.float64)

        self.t = 0.0
```

```
        self.history = {"t": [], "q": [], "omega": []}
        self._log()

    def _state_derivative(self, q, omega, torque):
        q_dot = self.kinematics.quaternion_dot(q, omega)
        omega_dot = self.dynamics.omega_dot(omega, torque)
        return q_dot, omega_dot

    def step(self, torque=np.zeros(3)):
        dt = self.dt
        q, omega = self.q, self.omega

        # RK4 implementation
        k1_q, k1_w = self._state_derivative(q, omega, torque)
        k2_q, k2_w = self._state_derivative(q + 0.5*dt*k1_q, omega +
            0.5*dt*k1_w, torque)
        k3_q, k3_w = self._state_derivative(q + 0.5*dt*k2_q, omega +
            0.5*dt*k2_w, torque)
        k4_q, k4_w = self._state_derivative(q + dt*k3_q, omega + dt*
            k3_w, torque)

        q_new = q + (dt/6.0)*(k1_q + 2*k2_q + 2*k3_q + k4_q)
        omega_new = omega + (dt/6.0)*(k1_w + 2*k2_w + 2*k3_w + k4_w)

        # re-normalize quaternion every step
        q_new = q_new / np.linalg.norm(q_new)

        self.q = q_new
        self.omega = omega_new
        self.t += dt
        self._log()

    def _log(self):
        self.history["t"].append(self.t)
        self.history["q"].append(self.q.copy())
        self.history["omega"].append(self.omega.copy())

    def run(self, duration, torque_func=None):
        steps = int(duration / self.dt)
        for _ in range(steps):
            torque = torque_func(self.t, self.q, self.omega) if
                torque_func else np.zeros(3)
            self.step(torque)
```

5 Usage Example

```
import numpy as np
from RK4_Propogator import AttitudeSimulator

# Example diagonal inertia tensor (kg m^2), typical of a 1U-3U CubeSat
I = np.diag([0.0033, 0.0033, 0.0008])

# Start aligned with the inertial frame, spinning slowly about x
q0 = [1.0, 0.0, 0.0, 0.0]
omega0 = [0.05, 0.0, 0.0]

```

```
sim = AttitudeSimulator(inertia_tensor=I, q0=q0, omega0=omega0, dt
    =0.01)

# Free (torque-free) tumble for 10 seconds
sim.run(duration=10.0)

# Access logged history for plotting
t_history = sim.history["t"]
q_history = sim.history["q"]
omega_history = sim.history["omega"]
```

A closed-loop controller can be passed in as `torque_func`, e.g. a PID law that computes a control torque from the quaternion attitude error and the angular velocity:

```
def pid_torque(t, q, omega):
    # ... compute attitude error from q, derive torque command ...
    return torque_command

sim.run(duration=30.0, torque_func=pid_torque)
```

6 Implementation Notes & Caveats

- **Torque is held constant per step.** All four RK4 stages within a single `step()` call use the same torque value, evaluated once at the start of the step in `run()`. For a controller running at the same rate as the dynamics this is standard practice, but it means torque is *not* re-sampled at the RK4 sub-stage times $t + h/2$ and $t + h$.
- **Quaternion renormalization is unconditional.** Every call to `step()` renormalizes q, regardless of how small the accumulated drift is. This is cheap (one norm and one division) and keeps the simulation numerically well-behaved over long integration horizons.
- **Fixed timestep only.** The integrator does not perform adaptive step-size control or local error estimation; `dt` should be chosen small enough relative to the fastest dynamics (e.g. the highest expected ω) to keep truncation error acceptable.
- **History storage.** `history["q"]` and `history["omega"]` store independent `.copy()` arrays per step, so the lists can be safely indexed and reused later (e.g. by `AttitudeVisualizer`) without aliasing issues.